PRD: Kill Chain and Attack Path Analysis — Problem, Goals and Requirements

30 October 2026 · Ratnadeep Dey Roy

1. Overview and problem statement

Build a tool that reads a large pile of security logs, finds the attack hidden inside, and shows it as one clear story: where the attacker started, what they did, and where they can go next.

The setting. Imagine a company with a few hundred laptops, servers and users. Every device writes logs. An attacker has broken in: they guessed a password, got onto a laptop, moved to a file server and copied data out. Each step left one line in a log, but those lines are spread across different files, millions of lines apart.

The problem. Security tools raise alerts, but only one at a time. So:

- **There is no complete story:** "failed login", "new admin account" and "big upload" look like three separate alerts, not one attack.
- **Manual linking is slow:** linking events manually takes an expert hours, while the attacker keeps moving.
- **Missed warnings:** nobody can tell if an alert is noise or step three of a break-in, and mapping to MITRE ATT&CK is done by hand, after the fact.

Your challenge. Build a tool that takes a large log data set, detects suspicious activity, maps it to MITRE ATT&CK techniques and kill chain stages, and rebuilds the attack path across hosts, users, accounts and time. Show what happened and what the attacker could reach next. Teams use the same data set as the Log Parser problem and pick one of three sizes (see section 4). You can choose any approach and any tech stack.

2. Goals and non-goals

A good solution finds the planted attacks, labels each step correctly, links the steps into a path, and finishes on a big data set without running out of memory.

Goals

- **Detect:** find the suspicious events hidden in the logs.
- **Label:** tag each event with a MITRE ATT&CK technique and a kill chain stage.
- **Connect:** link related events into one attack across hosts, users and time to create an attack path story
- **Explain:** show the attack as a timeline and a path that an analyst understands in a minute.
- **Look ahead:** show what the attacker could reach next.
- **Accurate:** results match the planted attacks (ground truth) with few false alarms.
- **Scales:** handles 15 GB or more of logs on a normal machine.

Not required

- A polished UI (a simple web page, a notebook or a graph tool is fine).
- Login, user management or cloud deployment.
- Blocking or stopping the attack. You only find and explain it.

What you will learn: log processing at scale, detection rules, MITRE ATT&CK and the kill chain, graph building, and how to measure detection accuracy.

3. Functional requirements

Your tool must turn raw logs into labelled, linked attack paths for any incident in the data set; the bonus items earn extra points.

Must have

ID	Feature	Example
FR-1	Read the full log data set without loading it all into memory (you can reuse your Log Parser)	Process the 15 GB tier on a 16 GB laptop
FR-2	Detect suspicious events	50 failed logins on one account, then a success
FR-3	Map each detection to a MITRE ATT&CK technique and a kill chain stage	"Brute Force (T1110)" → Credential Access
FR-4	Link related events into one incident by host, user, IP and time	Failed logins + new admin account + upload to an outside IP = one attack
FR-5	Show the attack path as a timeline and as a graph of hosts and users	Laptop HR-07 → file server FS-02 → domain controller DC-01
FR-6	Show what the attacker could reach next	"This account has admin rights on 3 more servers"
FR-7	Simple API or page, with a short README	GET /incidents/12/path

Bonus (pick any)

ID	Feature
B-1	Risk score for each incident, so the worst one is on top
B-2	Auto-written incident report in plain English
B-3	Show how many raw alerts were merged into each incident
B-4	Add a new detection rule by editing a config file (e.g. Sigma), not code
B-5	Live mode: new log lines update the attack path within seconds

4. Data set, performance targets and setup

Each team picks one data set size before starting; hit the Good column to qualify and the Great column to stand out.

Data set tiers (shared with the Log Parser problem; sizes are uncompressed)

Tier	Size	What it tests	Score multiplier
Core	15 GB	Bigger than a typical laptop's memory, so you must stream and process in chunks	1.0x
Advanced	30 GB	Needs parallel processing and compact storage to finish in time	1.3x
Extreme	50 GB	Full scale; a multi-core machine or cloud instance is expected	1.6x

- **Same attacks at every size:** every tier has the same formats and the same planted attack sequences (ground truth), so results are comparable.
- **Accuracy gate:** a bigger tier earns more only if accuracy stays above the minimum bar. Fast but sloppy does not win.
- **Dev sample:** a 100 MB sample is shared at kickoff for building. Full tiers are downloaded when the build starts.

Targets (measured against the planted attacks)

What we measure	How	Good	Great
Detection	Share of planted attack steps your tool finds	Over 70%	Over 90%
Correct labels	Share of found steps with the right ATT&CK technique	Over 60%	Over 85%
Attack path	Planted attacks rebuilt with the right steps in the right order	Half of them	All of them
False alarms	Share of your incidents that are not real attacks	Under 30%	Under 10%
Processing time	Time to process your full chosen tier	Under 3 hours	Under 1 hour
Memory	Peak memory used while processing	Under 16 GB	Under 8 GB
Investigation speed	Time to open one incident's attack path	Under 2 seconds	Under 500 ms

Suggested tools (not mandatory): Python, Go or Rust for parsing; DuckDB, ClickHouse or Elasticsearch for storage and search; Neo4j or NetworkX for the attack graph; Sigma rules and the MITRE ATT&CK data set for detection and labels.

Must run easily: judges should be able to start it by following the README (Docker Compose is a plus).